



KWARA STATE GOVERNMENT · L-PRES PROJECT

Technical Documentation

Kwara L-PRES Official Website
Architecture, Codebase & Developer Guide

VERSION

1.0.0

DATE

June 2026

STATUS

Live & Deployed

FRONTEND

React + Vite

BACKEND

FastAPI + SQLite

HOSTING

Netlify

Target audience: Junior Developers & Project Maintainers

Live URL: <https://kwara-lpres.netlify.app>

Table of Contents

1 Project Overview

What is L-PRES Website?

Technology Stack at a Glance

High-Level Architecture Diagram

2 Repository Structure

Root Layout

Backend File Map

Frontend File Map

Netlify Functions

3 Backend Deep Dive (FastAPI)

How the Backend Starts

Database Layer — SQLAlchemy + SQLite

Data Models

Pydantic Schemas

Authentication System (JWT)

API Routers — News, Gallery, Projects, Admin

File Upload Pattern

Database Migrations

4 Frontend Deep Dive (React + Vite)

App Entry Point & Routing

Context Providers

API Client Layer

Page Components

Section Components

Admin Panel

Styling System

Scroll Animation System

5 How Frontend and Backend Sync

The Request Lifecycle

Environment Variables & API_BASE

Multipart Form Submissions

Auth Token Flow

Media URL Resolution

6 Netlify Deployment

Why Netlify?

netlify.toml Explained

Serverless Functions

Static Assets Strategy

Deploying a New Build

7 Running Locally

Prerequisites

Starting the Backend

Starting the Frontend

Common Gotchas

8 Key Design Decisions

Why ?? (nullish coalescing) over || (OR) for API_BASE

Why Netlify Functions mirror the FastAPI schema

Multi-image gallery pattern

News section constraint

9 API Reference

News Endpoints

Gallery Endpoints

Projects Endpoints

Admin Auth Endpoints

10 Glossary & Junior Developer Tips

Project Overview

1.1 What is the L-PRES Website?

The **Kwara State Livestock Productivity and Resilience Support (L-PRES)** website is the official public-facing web platform for the Kwara State L-PRES Project — a World Bank-supported agricultural development initiative covering all 16 Local Government Areas of Kwara State, Nigeria.

The website serves two audiences simultaneously:

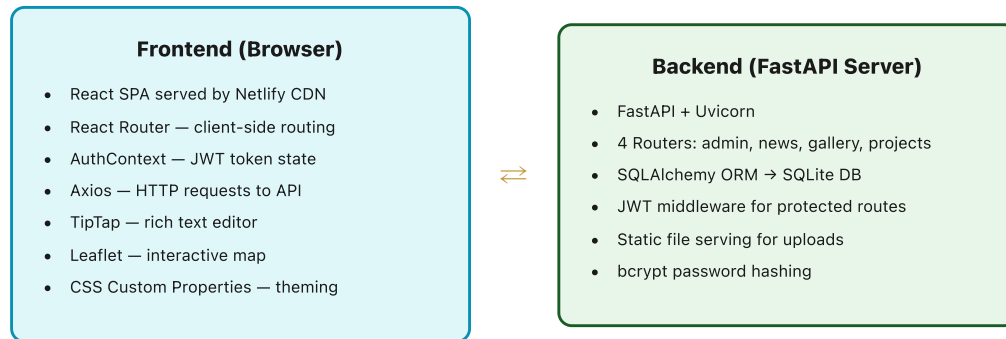
- **Public visitors** — citizens, journalists, development partners who browse news, project sites, gallery and contact information.
- **Administrators** — project staff who log in to publish news articles, manage the media gallery, and update project portfolio entries.

1.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend Framework	React	19.x	UI components and interactivity
Build Tool	Vite	8.x	Fast dev server and production bundler
Routing	React Router DOM	7.x	Client-side page navigation (SPA)
HTTP Client	Axios	1.x	API calls from the browser
Rich Text Editor	TipTap	3.x	WYSIWYG news article body editor
Map Library	Leaflet	1.9.x	Interactive field survey map on hero
Icons	Lucide React	1.x	Consistent SVG icon set throughout UI
Toast Notifications	React Hot Toast	2.x	Success / error feedback for admin actions
Backend Framework	FastAPI	0.104	REST API, file uploads, auth
ASGI Server	Uvicorn	0.24	Runs FastAPI in production / development
ORM	SQLAlchemy	2.0.x	Python → database mapping
Database	SQLite	3.x	Lightweight file-based relational database
Validation	Pydantic	v2	Request/response data validation
Auth	JWT (python-jose) + bcrypt	—	Stateless admin authentication
Async File I/O	aiofiles	23.x	Non-blocking file writes for uploads
Hosting	Netlify	—	CDN + Serverless Functions for static preview

1.3 High-Level Architecture

The system has two distinct runtime environments:



IMPORTANT NOTE — TWO MODES

In **local development**, the React app talks directly to <http://localhost:8000> (the real FastAPI server).
On **Netlify (live preview)**, there is no Python server — instead, three Netlify Functions (JavaScript) serve static JSON that mirrors the FastAPI response format exactly. The frontend code is identical in both cases.

Repository Structure

2.1 Root Layout

```
LPRES-WEBSITE/ # Project root |— backend/ # FastAPI Python application |— frontend/ # React +
Vite application |— netlify/ | |— functions/ # Serverless JS functions for Netlify preview |
|— projects.js | |— gallery.js | |— news.js |— netlify.toml # Netlify build + redirect
configuration
```

2.2 Backend File Map

```
backend/ |— main.py # App entry: creates FastAPI app, CORS, migrations, seed |— database.py #
SQLAlchemy engine, SessionLocal, Base, get_db() |— models.py # ORM table definitions (Admin,
NewsArticle, GalleryItem, Project) |— schemas.py # Pydantic models for request/response
validation |— auth.py # JWT creation/verification, bcrypt hashing, OAuth2 dependency |—
requirements.txt # Python package dependencies |— routers/ | |— __init__.py | |— admin.py #
POST /api/admin/login, /register, GET /me | |— news.py # Full CRUD for news articles + file
upload | |— gallery.py # Full CRUD for gallery items (photo + video) | |— projects.py # Full
CRUD for project portfolio + multi-image gallery |— uploads/ # Runtime folder – uploaded
images/videos stored here |— lpres_website.db # SQLite database file (created automatically)
```

2.3 Frontend File Map

```
frontend/ |— package.json # npm dependencies and scripts |— vite.config.js # Vite build
configuration |— public/ | |— hero-bg.jpg # Hero section background image | |— spc-photo.png
# Coordinator portrait | |— field-survey.json # GeoJSON data for the Leaflet map | |— uploads/
# Static copies of uploads for Netlify serving |— src/ |— main.jsx # React DOM entry point |—
App.jsx # Router, ScrollAnimator, ScrollToHash, route definitions |— index.css # Global CSS
variables, base styles, animation system |— api/ | |— client.js # Axios instance + all API
calls (newsAPI, galleryAPI, etc.) | |— context/ | |— AuthContext.jsx # JWT token state,
login/logout, useAuth() hook | |— ThemeContext.jsx# Light/dark theme toggle, useTheme() hook
|— components/ | |— layout/ # Navbar.jsx, Footer.jsx | |— sections/ # All homepage section
components (see below) | |— pages/ |— Home.jsx # Assembles all section components for the
homepage |— NewsPage.jsx # Paginated public news list |— NewsArticle.jsx # Single article view
(slug-based) |— GalleryPage.jsx # Filterable media gallery |— AboutPage.jsx # About the
project |— ProgramsPage.jsx# Programme components |— ResultsPage.jsx # Impact metrics |—
PartnersPage.jsx# Development partners |— ContactPage.jsx # Contact form |— admin/ |—
AdminLogin.jsx # Login form |— AdminDashboard.jsx # Stats overview + quick links |—
NewsForm.jsx # Create/edit news article with TipTap |— GalleryManager.jsx # Upload, edit,
delete gallery items |— ProjectsManager.jsx # Create/edit projects with multi-image gallery
```

2.4 Homepage Section Components

The homepage (`Home.jsx`) stacks these section components in order:

#	Component	What it Shows
1	<code>Hero.jsx</code>	Full-screen hero with YouTube documentary, photo carousel, and interactive Leaflet map
2	<code>QuickLinks.jsx</code>	Horizontal pill navigation bar linking to page sections
3	<code>CoordinatorMessage.jsx</code>	Coordinator portrait, quote and highlights
4	<code>Statistics.jsx</code>	Programme impact KPIs with animated counters
5	<code>ProjectComponents.jsx</code>	Five thematic project components
6	<code>ValueChains.jsx</code>	Cattle, sheep, goat, dairy value chain cards
7	<code>OnTheGround.jsx</code>	Project portfolio slider (fetches live API data)
8	<code>CoreValues.jsx</code>	Seven programme values grid
9	<code>NewsSection.jsx</code>	Latest news from API (FROZEN — do not modify)
10	<code>Partners.jsx</code>	Development partner logos and names
11	<code>Publications.jsx</code>	Reports and documents links
12	<code>Contact.jsx</code>	Contact form and office details

Backend Deep Dive (FastAPI)

3.1 How the Backend Starts

The backend is a single Python application in the `backend/` folder. To run it:

```
cd backend
source venv/bin/activate      # activate Python virtual environment
uvicorn main:app --reload     # start the dev server on http://localhost:8000
```

When `main.py` is executed, FastAPI does these things **in order**:

- 1. Create the uploads folder**
`os.makedirs("uploads", exist_ok=True)` — ensures the upload directory exists before any request arrives.
- 2. Create FastAPI app instance**
Sets title, version. Adds CORS middleware allowing requests from any `localhost` port.
- 3. Create database tables**
`models.Base.metadata.create_all(bind=engine)` reads all SQLAlchemy models and creates any missing tables.
- 4. Run migrations**
`run_migrations()` checks if the `images` column exists in the `projects` table. If not, it runs `ALTER TABLE` to add it.
- 5. Include routers**
Attaches `admin`, `news`, `gallery`, and `projects` routers — each prefixed with `/api/<name>`.
- 6. Seed default admin**
Creates username `admin` / password `lpres@admin2024` if no admin exists yet.
- 7. Seed sample data**
Inserts 5 sample projects and 3 gallery items if the tables are empty. Skips if data already exists.

3.2 Database Layer

`database.py` wires up SQLAlchemy:


```
SQLALCHEMY_DATABASE_URL = "sqlite:///lpres_website.db"

engine = create_engine(SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False}) # SQLite needs this for FastAPI
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base() # All models inherit from this

def get_db(): # FastAPI dependency — yields a DB session per request
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

JUNIOR DEVELOPER TIP

`get_db()` is a *dependency*. In any router function that needs the database, you write `db: Session = Depends(get_db)` as a parameter. FastAPI automatically calls `get_db()`, gives you the session, and closes it when the request is done — you never have to manage it manually.

3.3 Data Models (`models.py`)

Four database tables are defined as Python classes:

ADMIN

- `id` — primary key
- `username` — unique login name
- `email` — unique email
- `hashed_password` — bcrypt hash
- `is_active` — boolean flag
- `created_at` — auto timestamp

NEWSARTICLE

- `title`, `body` (HTML)
- `featured_image` — file URL
- `images` — JSON list of URLs
- `slug` — unique URL-friendly ID
- `template` — layout number (1–3)
- `is_published`, `category`

GALLERYITEM

- `file_url` — photo or video path
- `thumbnail_url` — for video posters
- `media_type` — "photo" or "video"
- `category`, `sort_order`
- `is_published`

PROJECT

- `name`, `lga`, `cluster`
- `description`, `status`
- `cover_image` — main photo URL
- `images` — JSON list (gallery)
- `highlights` — JSON list of strings
- `sort_order`, `is_published`

JSON COLUMNS

Columns like `images` and `highlights` use `Column(JSON)`. SQLAlchemy stores them as JSON text in SQLite and automatically parses them back into Python lists. **Never pass a JSON string to these columns** — always pass a Python list directly, or SQLAlchemy will double-encode it.

3.4 Pydantic Schemas (`schemas.py`)

Pydantic schemas define the *shape* of data going into and out of the API. They are separate from the models because the database structure and the API contract are not always identical (e.g., passwords

are stored hashed but never returned in responses).

Schema	Used for
<code>AdminCreate / AdminLogin</code>	Registration and login request bodies
<code>Token</code>	Login response — contains <code>access_token</code>
<code>NewsArticleCreate / Update / Out</code>	Create, update request bodies and response shape
<code>GalleryItemOut</code>	What the gallery API returns
<code>ProjectOut</code>	What the projects API returns

Every `Out` schema includes `class Config: from_attributes = True` — this tells Pydantic to read data from SQLAlchemy model attributes (not a dict).

3.5 Authentication System (`auth.py`)

The system uses **JWT (JSON Web Tokens)** for stateless authentication. Here is the full flow:

- 🕒 **Admin submits username + password**
Frontend POSTs to `/api/admin/login` as `application/x-www-form-urlencoded` (OAuth2 standard form).
- 🕒 **Backend verifies password**
`passlib` + `bcrypt` compares the plain password to the stored hash. If wrong, returns HTTP 401.
- 🕒 **Backend creates JWT**
`python-jose` signs a token with `{"sub": username, "exp": now + 24h}` using `ALGORITHM = "HS256"`.
- 🕒 **Frontend stores JWT in localStorage**
The Axios interceptor in `client.js` automatically attaches it as `Authorization: Bearer <token>` on every request.
- 🕒 **Protected endpoints verify the token**
`get_current_admin` dependency decodes the JWT, looks up the admin in DB, and either returns the admin object or raises HTTP 401.

SECURITY NOTE

The `SECRET_KEY` in `auth.py` must be changed before going to production on a real server. The current value is a placeholder. Use a randomly generated 64-character string.

3.6 API Routers

Each router is a separate file. FastAPI uses Python decorators to define endpoints:

```
@router.get("/", response_model=List[schemas.ProjectOut])
def list_projects(skip: int = 0, limit: int = 50, db: Session = Depends(get_db)):
    # FastAPI automatically converts the return value using ProjectOut schema
    return db.query(models.Project).filter(models.Project.is_published == True).all()
```

Public vs Admin Endpoints

Every router has two sets of endpoints. Public endpoints (no auth) return only published records. Admin endpoints require the `get_current_admin` dependency and return all records:

Pattern	Auth Required?	Example
<code>GET /api/news/</code>	No	List published articles
<code>GET /api/news/{slug}</code>	No	Get one published article
<code>GET /api/news/admin/all</code>	Yes (JWT)	List all articles including drafts
<code>POST /api/news/admin/create</code>	Yes (JWT)	Create article
<code>PUT /api/news/admin/{id}</code>	Yes (JWT)	Update article
<code>PATCH /api/news/admin/{id}/publish</code>	Yes (JWT)	Toggle publish status
<code>DELETE /api/news/admin/{id}</code>	Yes (JWT)	Delete article

3.7 File Upload Pattern

All three routers (news, gallery, projects) use the same upload pattern:

```
async def _save_image(file: UploadFile) -> str:
    # 1. Validate MIME type against whitelist
    if file.content_type not in ALLOWED_IMAGE:
        raise HTTPException(400, "Unsupported type")

    # 2. Read entire file into memory and check size
    content = await file.read()
    if len(content) > MAX_IMAGE_SIZE:
        raise HTTPException(400, "File too large")

    # 3. Generate a UUID filename to prevent collisions and path traversal
    ext = file.filename.rsplit(".", 1)[-1].lower()
    filename = f"{uuid.uuid4().hex}.{ext}"

    # 4. Write to disk asynchronously (doesn't block the server)
    async with aiofiles.open(os.path.join(UPLOAD_DIR, filename), "wb") as f:
        await f.write(content)

    # 5. Return a URL path the frontend can use to fetch the file
    return f"/api/projects/uploads/{filename}"
```

Files are served back via a static endpoint in each router:

```
@router.get("/uploads/{filename}")
def serve_upload(filename: str):
    path = os.path.join(UPLOAD_DIR, filename)
    if not os.path.exists(path):
        raise HTTPException(404, "File not found")
    return FileResponse(path)  # FastAPI streams the file to the browser
```

3.8 Multi-Image Project Gallery

Projects have a `cover_image` (one main photo) and `images` (a list for the gallery carousel). On update, the admin UI sends two extra fields:

- `keep_images` — JSON string listing existing image URLs to retain (those not listed are deleted from disk)
- `images` — new file uploads to append to the gallery

```
# Reconcile: delete dropped images, keep retained ones
if keep_images is not None:
    keep_list = json.loads(keep_images)
    for url in (p.images or []):
        if url not in keep_list:
            _delete_file(url) # physically removes from disk
    p.images = keep_list

# Append new uploads
if images:
    new_urls = [await _save_image(img) for img in images if img.filename]
    p.images = list(p.images or []) + new_urls
```

Frontend Deep Dive (React + Vite)

4.1 App Entry Point & Routing (`App.jsx`)

`src/main.jsx` is the true entry point — it mounts `<App />` into the HTML. `App.jsx` sets up the entire routing and global infrastructure:

```
export default function App() {
  return (
    <BrowserRouter>           {/* enables URL-based navigation */}
    <ThemeProvider>           {/* wraps everything in theme context */}
    <AuthProvider>             {/* wraps everything in auth context */}
    <Toaster />                 {/* toast notification outlet */}
    <ScrollToHash />           {/* scrolls to #anchor on route change */}
    <ScrollAnimator />         {/* triggers scroll animations */}
    <Routes>
      <Route path="/" element={<PublicLayout><Home /></PublicLayout>} />
      {/* ... more public routes ... */}
      <Route path="/admin/dashboard"
        element={<AdminRoute><AdminDashboard /></AdminRoute>} />
    </Routes>
    </AuthProvider>
    </ThemeProvider>
    </BrowserRouter>
  );
}
```

Route Protection

`AdminRoute` is a wrapper that checks authentication before rendering admin pages:

```
function AdminRoute({ children }) {
  const { isAuthenticated, loading } = useAuth();
  if (loading) return <div style={{ minHeight: '100vh' }} />; // show blank while checking
  return isAuthenticated ? children : <Navigate to="/admin/login" replace />;
}
```

Scroll Animator

A lightweight IntersectionObserver-based animation system. No external library needed:

```
function ScrollAnimator() {
  const { pathname } = useLocation();
  useEffect(() => {
    const observer = new IntersectionObserver(
      (entries) => entries.forEach(e => {
        if (e.isIntersecting) {
          e.target.classList.add('in-view'); // triggers CSS transition
          observer.unobserve(e.target); // animate only once
        }
      })
    ),
    { threshold: 0.12, rootMargin: '0px 0px -30px 0px' }
  );
  const timer = setTimeout(() => {
    // re-scan for new [data-animate] elements after route renders
    document.querySelectorAll('[data-animate]:not(.in-view)')
      .forEach(el => observer.observe(el));
  }, 80);
  return () => { clearTimeout(timer); observer.disconnect(); };
}, [pathname]);
return null; // renders nothing visible
}
```

To animate any element, add `data-animate` to it. Use `data-delay="2"` through `"6"` to stagger siblings. The CSS in `index.css` handles the rest.

4.2 Context Providers

AuthContext (`context/AuthContext.jsx`)

Manages the JWT token lifecycle and exposes a `useAuth()` hook:

```
// In any component:
const { admin, isAuthenticated, login, logout } = useAuth();

// login() calls the API, stores the token, then fetches /api/admin/me
// logout() clears localStorage and resets admin to null
```

On first render, if a token exists in `localStorage`, the context calls `/api/admin/me` to verify it is still valid before setting `isAuthenticated = true`.

ThemeContext (`context/ThemeContext.jsx`)

Controls dark/light mode by toggling a `data-theme` attribute on `<html>`. CSS custom properties in `index.css` respond to this attribute.

4.3 API Client Layer (`api/client.js`)

A single Axios instance is shared across all components. It has two interceptors:

REQUEST INTERCEPTOR

Before every request: reads `lpres_admin_token` from `localStorage` and appends it as `Authorization: Bearer ...` header. If no token, nothing is added (public routes work without it).

RESPONSE INTERCEPTOR

On every response: if status is 401 (Unauthorized), clears the token from `localStorage` and redirects to `/admin/login`. Handles expired tokens automatically.

API calls are grouped by resource:

```
export const newsAPI = {
  list: (skip, limit) => api.get(`/api/news/?skip=${skip}&limit=${limit}`),
  get: (slug) => api.get(`/api/news/${slug}`),
  create: (formData) => api.post('/api/news/admin/create', formData,
    { headers: { 'Content-Type': 'multipart/form-data' } }),
  // ...
};

export const projectsAPI = { /* same pattern */ };
export const galleryAPI = { /* same pattern */ };
export const adminAPI = { /* login + me */ };
```

JUNIOR DEVELOPER TIP

Always use these named API objects in your components, never construct raw `axios.get()` calls. This gives you one place to change URLs or headers if the backend changes.

4.4 The OnTheGround Component — Live Data Fetching

This is the best example of how a section component fetches and displays real backend data:

```
export default function OnTheGround() {
  const [apiProjects, setApiProjects] = useState([]);

  useEffect(() => {
    fetch(`${API_BASE}/api/projects/?limit=20`)
      .then(r => r.json())
      .then(data => {
        if (Array.isArray(data) && data.length > 0) setApiProjects(data);
        // If empty or error, apiProjects stays [] and FALLBACK data is used
      })
      .catch(() => {}); // silently ignore network errors
  }, []);

  const displayProjects = apiProjects.length > 0 ? apiProjects : FALLBACK;
  // ... render using displayProjects
}
```

4.5 Admin Panel

The admin section lives in `pages/admin/` and is only accessible after login.

NewsForm.jsx — TipTap Rich Text Editor

The news article body is written using TipTap, a headless rich-text editor built on ProseMirror. It outputs standard HTML that gets stored in the `body` column and rendered in news article templates using `dangerouslySetInnerHTML`.

ProjectsManager.jsx — Multi-image Gallery

Uses a two-part state pattern for managing the project's image gallery:

```
const [keepImages, setKeepImages] = useState(project?.images || []);
const [extraFiles, setExtraFiles] = useState([]); // new files not yet uploaded

// On submit, build a FormData:
const fd = new FormData();
fd.append('keep_images', JSON.stringify(keepImages)); // tells backend which to keep
extraFiles.forEach(f => fd.append('images', f)); // new uploads
```

The UI shows existing thumbnails with an X button to remove them, and a file zone to add new photos.

4.6 Styling System

The site uses plain CSS with custom properties (variables). No CSS framework like Tailwind is used.

Global Variables (defined in `index.css`)

```
:root {
  --bg-primary: #0a0f0d; /* dark background */
  --gold-primary: #c49a3e; /* L-PRES brand gold */
  --green-deep: #1a5c26; /* deep forest green */
  --text-primary: rgba(255,255,255,0.92);
  --text-muted: rgba(255,255,255,0.55);
}
```

Responsive Grid Pattern

Most grids use CSS Grid with `auto-fill` and `minmax()` for automatic responsiveness:

```
.cv__grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));
  gap: 20px;
}
```

4.7 Scroll Animation System

Piece	Location	What it does
<code>ScrollAnimator</code> component	<code>App.jsx</code>	IntersectionObserver that watches <code>[data-animate]</code> elements and adds <code>.in-view</code> when visible
<code>[data-animate]</code> CSS	<code>index.css</code>	Sets initial state: <code>opacity:0; transform:translateY(22px)</code>
<code>[data-animate].in-view</code> CSS	<code>index.css</code>	Target state: <code>opacity:1; transform:none</code> — CSS handles the smooth transition
<code>data-delay="N"</code> attribute	JSX on any element	Adds <code>N * 0.08s</code> transition delay for staggered reveals
Hero keyframes	<code>Hero.css</code>	Above-the-fold elements use CSS <code>@keyframes</code> with <code>animation-fill-mode: both</code>
<code>prefers-reduced-motion</code>	<code>index.css</code> + <code>Hero.css</code>	Disables all animations for users who request reduced motion

How Frontend and Backend Sync

5.1 The Request Lifecycle (end-to-end)

Here is what happens when a visitor loads the homepage and the Projects section fetches data:

- Browser loads the React app**

Netlify CDN serves `frontend/dist/index.html` and the JS bundle. React hydrates the page.
- OnTheGround component mounts**

`useEffect` fires after the first render. It calls `fetch(`${API_BASE}/api/projects/?limit=20`)`.
- Request hits Netlify redirect**

Because `VITE_API_URL=""`, `API_BASE` is an empty string, so the URL is `/api/projects/` — a relative URL. Netlify's `netlify.toml` redirect rule intercepts it.
- Netlify Function handles the request**

`netlify/functions/projects.js` returns the hardcoded JSON array of 5 projects. In local development, this step is replaced by the real FastAPI server at `localhost:8000`.
- React updates state**

`setApiProjects(data)` triggers a re-render. The project cards display with the real data.
- Image URLs are resolved**

`mediaUrl(url)` checks if the URL starts with `http`. If it's a relative path like `/uploads/seed_hero.jpg`, it is prefixed with `API_BASE` to form the full URL.

5.2 The Critical ?? vs || Distinction

This is one of the most important technical decisions in the codebase. Every non-news file uses:

```
const API_BASE = import.meta.env.VITE_API_URL ?? 'http://localhost:8000';
```

Operator	Falls back when value is...	Behaviour with <code>VITE_API_URL=""</code>
<code>??</code> (nullish coalescing)	<code>null</code> or <code>undefined</code> only	✅ Uses <code>""</code> → relative URLs → Netlify redirects work
<code> </code> (logical OR)	Any falsy value: <code>null</code> , <code>undefined</code> , <code>""</code> , <code>0</code> , <code>false</code>	❌ Falls back to <code>localhost:8000</code> → broken on Netlify

DO NOT CHANGE

`NewsSection.jsx` and all news template files intentionally still use `||`. They are not to be modified. The `netlify/functions/news.js` compensates by returning absolute Unsplash URLs for images, so the concatenation produces a valid URL regardless.

5.3 Multipart Form Submissions

File uploads cannot be sent as JSON. They must use `multipart/form-data`. Here is the full pattern from the admin to the backend:

```
// Frontend (ProjectsManager.jsx)
const fd = new FormData();
fd.append('name', formData.name); // text field
fd.append('lga', formData.lga);
fd.append('highlights', JSON.stringify(highlightsList)); // JSON as string
fd.append('keep_images', JSON.stringify(keepImages)); // JSON as string
if (coverFile) fd.append('cover_image', coverFile); // binary file
extraFiles.forEach(f => fd.append('images', f)); // multiple files

// Axios sends it with correct Content-Type
await projectsAPI.update(id, fd);
```

```
# Backend (projects.py) reads each field by name:
@router.put("/admin/{project_id}")
async def update_project(
    project_id: int,
    name: Optional[str] = Form(None), # text
    highlights: Optional[str] = Form(None), # JSON string → parsed with json.loads()
    keep_images: Optional[str] = Form(None), # JSON string
    cover_image: Optional[UploadFile] = File(None), # file
    images: Optional[List[UploadFile]] = File(default=None), # multiple files
    ...
):
```

5.4 Auth Token Flow

```
// 1. Login – AdminLogin.jsx
const { login } = useAuth();
await login(username, password); // AuthContext calls adminAPI.login()

// 2. adminAPI.login() in client.js
const form = new URLSearchParams();
form.append('username', username);
form.append('password', password);
return api.post('/api/admin/login', form,
    { headers: { 'Content-Type': 'application/x-www-form-urlencoded' } });
// FastAPI expects this exact format for OAuth2PasswordRequestForm

// 3. Token stored in AuthContext
localStorage.setItem('lpres_admin_token', res.data.access_token);

// 4. Interceptor attaches it automatically
api.interceptors.request.use((config) => {
    const token = localStorage.getItem('lpres_admin_token');
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});
```

5.5 Media URL Resolution

Every component that displays uploaded images uses this helper function:

```
const API_BASE = import.meta.env.VITE_API_URL ?? 'http://localhost:8000';

function mediaUrl(url) {
  if (!url) return null;
  // If url already starts with http, use as-is (absolute URL from Netlify function)
  // If url is a path like /api/projects/uploads/abc.jpg, prepend API_BASE
  return url.startsWith('http') ? url : `${API_BASE}${url}`;
}
```

This handles both environments transparently:

Environment	API_BASE	Path from DB	Final URL
Local dev	http://localhost:8000	/api/projects/uploads/abc.jpg	http://localhost:8000/api/projects/uploads/i
Netlify	""	/uploads/abc.jpg	/uploads/abc.jpg (served from Netlify CDN)

Netlify Deployment

6.1 Why Netlify for the Preview?

The full system (FastAPI + SQLite) requires a Python server that cannot run on Netlify's static CDN. For the client preview site, Netlify was chosen because:

- Free hosting with global CDN (sub-100ms load times)
- Automatic HTTPS
- Netlify Functions allow simple JavaScript serverless functions to mimic the FastAPI responses
- One-command deployments: `npx netlify deploy --prod`

6.2 netlify.toml Explained Line by Line

```
[build]
  command = "cd frontend && npm run build" # Vite builds the React app
  publish = "frontend/dist"               # Serve from this output folder

[functions]
  directory = "netlify/functions"         # JS serverless functions location

[build.environment]
  VITE_API_URL = ""                       # Empty string → relative API URLs

# Each [[redirects]] block maps a URL pattern to a function
[[redirects]]
  from = "/api/projects"                 # Match this URL path
  to   = "/.netlify/functions/projects"  # Forward to this function
  status = 200                           # 200 = proxy (not a real redirect)

[[redirects]]
  from = "/api/projects/*"               # Also match /api/projects/1, /api/projects/anything
  to   = "/.netlify/functions/projects"
  status = 200

# ... same for /api/gallery/* and /api/news/*

[[redirects]]
  from = "/*"                            # MUST be last – SPA catch-all
  to   = "/index.html"                   # All unmatched URLs → React app
  status = 200
```

REDIRECT ORDER MATTERS

API redirects must come **before** the SPA catch-all (`/*`). If the catch-all were first, every `/api/projects` request would return `index.html` instead of the JSON data, and the whole site would break.

6.3 Netlify Serverless Functions

Each function in `netlify/functions/` is a JavaScript file that exports a `handler` function. It receives the HTTP request and returns a response object:

```
// netlify/functions/projects.js (simplified)
exports.handler = async (event) => {
  const headers = {
    "Content-Type": "application/json",
    "Access-Control-Allow-Origin": "*",
  };

  // Check if a specific ID was requested (e.g., /api/projects/2)
  const path = event.path.replace(/^\/api\/projects\/?/, "");
  const id = path ? parseInt(path, 10) : null;

  if (id) {
    const project = PROJECTS.find(p => p.id === id);
    if (!project) return { statusCode: 404, headers, body: JSON.stringify({ detail: "Not found" }) };
    return { statusCode: 200, headers, body: JSON.stringify(project) };
  }

  // Return all projects
  return { statusCode: 200, headers, body: JSON.stringify(PROJECTS) };
};
```

The `PROJECTS` array at the top of the file mirrors the exact shape of `schemas.Project00t` from the FastAPI backend. This is critical — if the shapes don't match, the frontend breaks.

6.4 Static Assets on Netlify

Uploaded photos stored in `backend/uploads/` during local development are not accessible on Netlify (there is no Python server). Instead, copies were placed in `frontend/public/uploads/`:

```
# Files placed in frontend/public/uploads/:
seed_hero.jpg      # used by Netlify functions as cover images
seed_spc.png       # coordinator photo
seed_video.mp4     # sample video in gallery
```

Vite copies everything in `public/` to `dist/` at build time, so these are served as static files by Netlify CDN at `/uploads/filename.jpg`.

6.5 Deploying a New Build

```
# From the project root (LPRES-WEBSITE/):
npx netlify deploy --prod

# This command:
# 1. Runs: cd frontend && npm run build (from netlify.toml)
# 2. Bundles netlify/functions/*.js
# 3. Uploads changed files to Netlify CDN
# 4. Makes the deploy live at https://kwara-lpres.netlify.app
```

FIRST-TIME SETUP

If deploying from a new machine: first run `npx netlify login` to authenticate, then `npx netlify link` from the project root to link to the existing site. The `.netlify/state.json` file stores the site ID for subsequent deploys.

Running Locally

7.1 Prerequisites

Tool	Minimum Version	Install
Python	3.10+	python.org
Node.js	18+	nodejs.org
npm	9+	Bundled with Node.js
Git	Any	Pre-installed on most systems

7.2 Starting the Backend

```
# Step 1: Navigate to backend folder
cd backend

# Step 2: Create a Python virtual environment (first time only)
python3 -m venv venv

# Step 3: Activate it
# macOS / Linux:
source venv/bin/activate
# Windows:
venv\Scripts\activate

# Step 4: Install dependencies (first time only)
pip install -r requirements.txt

# Step 5: Start the server
uvicorn main:app --reload --port 8000

# The API is now available at:
# http://localhost:8000      - API root
# http://localhost:8000/docs - Interactive Swagger UI
# http://localhost:8000/health - Health check
```

AUTO SEED

The first time you run the backend, it automatically creates the database, adds the default admin (`admin / lpres@admin2024`), and seeds 5 sample projects + 3 gallery items. You do not need to run any migration commands manually.

7.3 Starting the Frontend

```
# Open a NEW terminal window/tab (keep backend running)

# Step 1: Navigate to frontend folder
cd frontend

# Step 2: Install dependencies (first time only)
npm install

# Step 3: Start the Vite dev server
npm run dev

# The site is now at http://localhost:5173
# (or another port if 5173 is busy – Vite will tell you)
```

Vite's dev server has Hot Module Replacement (HMR) — any file you edit instantly updates in the browser without a page reload.

7.4 Environment Variables

In local development, no `.env` file is needed. The default fallback in `client.js` (`http://localhost:8000`) is used automatically when `VITE_API_URL` is not set.

If you need to override the API URL, create `frontend/.env.local`:

```
VITE_API_URL=http://localhost:8000
```

DO NOT SET VITE_API_URL="" LOCALLY

An empty string works on Netlify because relative URLs route through Netlify redirects. Locally, relative URLs go to the Vite dev server (port 5173), not the FastAPI server (port 8000). Leave `VITE_API_URL` unset for local development.

7.5 Common Gotchas

Problem	Cause	Fix
CORS error in browser console	Frontend on port 5173, backend on 8000, but CORS only allows <code>localhost:*</code>	Backend CORS uses regex <code>r"http://localhost:\d+"</code> — any localhost port is allowed. Check the backend is actually running.
Login fails with 401	Wrong credentials or database recreated	Default admin: <code>admin</code> / <code>lpres@admin2024</code> . If DB was deleted, it is re-seeded on next start.
Images don't show	Backend not running / uploads folder missing	Ensure backend is on port 8000 and <code>backend/uploads/</code> exists. The folder is created automatically on startup.
bcrypt error on startup	<code>bcrypt==5.x</code> breaks passlib	<code>requirements.txt</code> pins to <code>bcrypt==4.0.1</code> . Run <code>pip install -r requirements.txt</code> to enforce this.
Port already in use	Previous server still running	Kill the process: <code>lsof -ti:8000 xargs kill</code> (macOS/Linux) or use Task Manager on Windows.

Key Design Decisions

8.1 Why ?? (nullish coalescing) instead of || ?

The Netlify build sets `VITE_API_URL=""` (an empty string). Empty strings are falsy in JavaScript. Using `||` would fall back to `'http://localhost:8000'` even in production. Using `??` only falls back for `null` or `undefined`, so `""` is used as-is, making all API calls relative — which Netlify then routes to the Functions.

8.2 Why Netlify Functions Instead of a Real Server?

The client wanted a shareable preview URL immediately, without setting up server infrastructure. Netlify Functions are zero-configuration JavaScript that run on AWS Lambda under the hood. They serve static JSON matching the FastAPI schema shape. When a real production server is deployed, only `VITE_API_URL` needs updating — the frontend code changes nothing.

8.3 Multi-image Pattern: keep_images + images

When editing a project's gallery, the admin can remove existing photos (by not listing them in `keep_images`) and add new ones (via file upload). Sending only the *list to keep* (not a list to delete) means the backend makes one pass through existing images, compares to the keep list, and physically deletes orphaned files from disk. No separate "delete image" API endpoint is needed.

8.4 NewsSection — The Frozen Component

`NewsSection.jsx` and all news display templates are intentionally never modified. They use `||` for `API_BASE` (not `??`) and concatenate `API_BASE` with image paths. The Netlify news function compensates by returning absolute Unsplash image URLs, which the `startsWith('http')` check in the templates recognises as complete URLs.

8.5 SQLite for Storage

SQLite is a file-based database (one `.db` file) with no server process required. This is a good fit for a single-server deployment with low concurrent write load. If the project scales or needs multiple server instances, migrating to PostgreSQL with SQLAlchemy requires only changing the `SQLALCHEMY_DATABASE_URL` in `database.py` — all model and router code stays the same.

Complete API Reference

Base URL (local): `http://localhost:8000` · Interactive docs: `http://localhost:8000/docs`

9.1 Authentication Endpoints

Method	Path	Auth	Description
POST	<code>/api/admin/login</code>	—	Login — returns JWT token. Body: <code>x-www-form-urlencoded</code> with <code>username</code> + <code>password</code>
GET	<code>/api/admin/me</code>	JWT	Returns current admin profile
POST	<code>/api/admin/register</code>	—	Create new admin account

9.2 News Endpoints

Method	Path	Auth	Description
GET	<code>/api/news/</code>	—	List published articles. Query: <code>skip</code> , <code>limit</code>
GET	<code>/api/news/{slug}</code>	—	Get one article by slug
GET	<code>/api/news/category/{cat}</code>	—	Filter by category
GET	<code>/api/news/uploads/{file}</code>	—	Serve uploaded image file
GET	<code>/api/news/admin/all</code>	JWT	List all articles (including drafts)
POST	<code>/api/news/admin/create</code>	JWT	Create article. Body: <code>multipart/form-data</code>
PUT	<code>/api/news/admin/{id}</code>	JWT	Update article. Body: <code>multipart/form-data</code>
PATCH	<code>/api/news/admin/{id}/publish</code>	JWT	Toggle <code>is_published</code>
DELETE	<code>/api/news/admin/{id}</code>	JWT	Delete article

9.3 Gallery Endpoints

Method	Path	Auth	Description
GET	/api/gallery/	—	List items. Query: <code>category</code> , <code>media_type</code> , <code>skip</code> , <code>limit</code>
GET	/api/gallery/categories	—	List categories with item counts
GET	/api/gallery/uploads/{file}	—	Serve uploaded media file
GET	/api/gallery/admin/all	JWT	All items including unpublished
POST	/api/gallery/admin/upload	JWT	Upload photo or video item
PUT	/api/gallery/admin/{id}	JWT	Update item metadata or replace file
PATCH	/api/gallery/admin/{id}/publish	JWT	Toggle is_published
DELETE	/api/gallery/admin/{id}	JWT	Delete item and its files from disk

9.4 Projects Endpoints

Method	Path	Auth	Description
GET	/api/projects/	—	List published projects. Query: <code>skip</code> , <code>limit</code>
GET	/api/projects/{id}	—	Get one project by ID
GET	/api/projects/uploads/{file}	—	Serve project image
GET	/api/projects/admin/all	JWT	All projects including unpublished
POST	/api/projects/admin/	JWT	Create project with images
PUT	/api/projects/admin/{id}	JWT	Update project. Supports <code>keep_images</code> + new <code>images</code>
PATCH	/api/projects/admin/{id}/publish	JWT	Toggle is_published
DELETE	/api/projects/admin/{id}	JWT	Delete project and all its images from disk

Glossary & Junior Developer Tips

10.1 Glossary

Term	What it means
SPA	Single Page Application. The browser loads one HTML file and React handles all navigation by swapping components — no full page reloads.
Vite	A build tool and dev server. Runs the React app in development, and bundles it into static files (<code>dist/</code>) for production.
FastAPI	Python web framework that auto-generates API docs, validates data with Pydantic, and is very fast due to async support.
SQLAlchemy	Python ORM (Object Relational Mapper). Write Python classes, it handles SQL queries. You rarely write raw SQL.
SQLite	A database that lives entirely in a single file (<code>lpres_website.db</code>). No separate database server needed.
Pydantic	Python data validation library. Defines the expected shape of API request and response bodies.
JWT	JSON Web Token. A signed string containing the admin's username and expiry. The server never stores sessions — it validates the signature on every request.
bcrypt	Password hashing algorithm. Takes a plain password, adds a random salt, and produces a hash. You cannot reverse a bcrypt hash to get the original password.
CORS	Cross-Origin Resource Sharing. Browser security rule that blocks requests from <code>localhost:5173</code> to <code>localhost:8000</code> unless the backend explicitly allows it.
Context (React)	A way to share state across many components without passing props through every layer. <code>AuthContext</code> makes login state accessible anywhere in the app.
useEffect	React hook that runs code after the component renders. Used for data fetching, subscriptions, and DOM interactions.
IntersectionObserver	Browser API that calls a function when an element enters or exits the viewport. Used here for scroll animations.
Multipart/form-data	HTTP encoding format required for file uploads. Cannot send binary files as JSON.
Netlify Functions	Serverless JavaScript functions that run on Netlify's CDN edge. Each function is one file and handles one group of API paths.
HMR	Hot Module Replacement. Vite's ability to update changed modules in the browser instantly during development without a page refresh.
Router (FastAPI)	An <code>APIRouter</code> groups related endpoints. Each router has a prefix (e.g., <code>/api/news</code>) so endpoint paths don't need to repeat it.
Dependency Injection	FastAPI pattern: <code>Depends(get_db)</code> means "call <code>get_db()</code> and pass its result as the <code>db</code> parameter". The framework handles calling it.
UUID	Universally Unique Identifier. Used as filenames for uploads to guarantee no two files ever have the same name, regardless of original filename.
slug	A URL-friendly version of a title. "My News Article" becomes <code>my-news-article</code> . Used in news URLs: <code>/news/my-news-article</code> .

10.2 Top Tips for Junior Developers

TIP 1 — READ THE SWAGGER DOCS FIRST

With the backend running, visit `http://localhost:8000/docs` . You can try every API endpoint interactively without writing any code. This is the fastest way to understand what data each endpoint expects and returns.

TIP 2 — ONE FILE PER CONCERN

The project strictly separates concerns: database config in `database.py` , table definitions in `models.py` , response shapes in `schemas.py` , auth logic in `auth.py` , and API routes in `routers/` . When you need to add a field, you will typically need to update models, schemas, AND the router.

TIP 3 — NEVER HARDCODE API URLS IN COMPONENTS

All API calls go through `api/client.js` . If you need a new endpoint, add a method to the appropriate API object (`newsAPI` , `galleryAPI` , etc.). Never write `axios.get('http://localhost:8000/...')` directly in a component.

TIP 4 — ADDING A NEW SECTION TO THE HOMEPAGE

Create `MySection.jsx` and `MySection.css` in `components/sections/` , then import and add it to `pages/Home.jsx` . Add `data-animate` to the section header element. No other configuration needed — the `ScrollAnimator` picks it up automatically.

TIP 5 — TEST BOTH ENVIRONMENTS

After making changes, always test locally (with the FastAPI backend running) AND on Netlify. Some bugs only appear in one environment — particularly anything involving API URLs or file paths.

CRITICAL — NEVER MODIFY NEWSSECTION.JSX

`frontend/src/components/sections/NewsSection.jsx` and all files in the news templates folder must NOT be edited. They use `||` for `API_BASE` by design, and the Netlify news function is tuned to compensate. Modifying these files will break the news display in unpredictable ways.

10.3 Where to Add Things

Task	File(s) to edit
Add a new database table	<code>backend/models.py</code> , then <code>schemas.py</code> , then a new router in <code>routers/</code>
Add a new API endpoint	The appropriate router file in <code>backend/routers/</code>
Add a new frontend API call	<code>frontend/src/api/client.js</code> — add method to the relevant API object
Add a new page	New file in <code>frontend/src/pages/</code> + add a <code><Route></code> in <code>App.jsx</code>
Add a homepage section	New component in <code>frontend/src/components/sections/</code> + import in <code>pages/Home.jsx</code>
Change brand colours	<code>frontend/src/index.css</code> — update the <code>:root { --gold-primary, --green-deep, ... }</code> variables
Add data to Netlify preview	<code>netlify/functions/projects.js</code> (or <code>gallery.js</code> / <code>news.js</code>)
Change Netlify build command	<code>netlify.toml</code> — the <code>command</code> field
Change admin default password	<code>backend/main.py</code> — <code>seed_default_admin()</code> function, then delete the DB and restart

Kwara State Livestock Productivity and Resilience Support (L-PRES) Project

Technical Documentation v1.0 · June 2026 · Confidential — For project team use

Live site: <https://kwara-lpres.netlify.app> · API docs (local): <http://localhost:8000/docs>